DOCUMENTO GUÍA DEL PROCESO DE DESARROLLO DE SOFTWARE

Proyecto: Sistema de Ventas de Aparatos Electrónicos de Escritorio | Estado: [BORRADOR]

Control de versiones

Nombre	Fecha	Razón del cambio	Autor	Versión	Hora Inicio	Hora Fin	Tiempo Estimado	Tiempo Real
STK_PDS_V0_1_130926.pdf	13-Sep-2026	Creación e inicialización del documento borrador de proceso.	DAOJ	0.1	12:00	18:30	6 hrs	6 hrs 30 min
STKD_PDS_V0_1_150926.pdf	14-Sep-2026	Adaptación inicial al programa de escritorio y definición de los dos procesos principales.	AESA	0.1	13:31	21:45	8 hrs	8 hrs 14 min
STKD_PDS_V0_2_150926.pdf	15-Sep-2026	Revisión del modelo incremental, las entregas por valor de negocio, los criterios y la trazabilidad.	AESA	0.2	09:00	18:17	10 hrs	9 hrs 17 min

1. Introducción

En esta guía dejamos por escrito cómo vamos a desarrollar el Sistema de Ventas de Aparatos Electrónicos de Escritorio de Empresa StockFlow. Será un programa instalado en computadoras. Lo usaremos mediante ventanas, formularios y menús, con teclado y mouse. No necesitará un navegador y tampoco se construirá como sitio web.

Trabajaremos con dos procesos principales. En el primero construiremos el sistema poco a poco, con incrementos pequeños que podamos probar y revisar. En el segundo nos ocuparemos de que el programa pueda crecer en datos, módulos y equipos sin tener que empezar de nuevo. También habrá tareas de calidad, cambios, configuración y documentación. Esas tareas acompañan a los dos procesos; no forman procesos principales aparte.

2. Objetivo y alcance

Objetivo. Queremos que el equipo siga una forma de trabajo clara, repetible y fácil de rastrear mientras construimos y ampliamos la aplicación de escritorio. Para lograrlo, avanzaremos por iteraciones y revisaremos la escalabilidad desde cada versión.

Dentro del alcance. Trabajaremos con catálogo de productos, existencias, carrito, cálculo del total, datos del cliente, compra simulada y generación de órdenes. También incluiremos almacenamiento persistente, instalación, actualización y respaldo en Windows 10 y Windows 11 de 64 bits.

Fuera del alcance. No construiremos un sitio web ni una versión para teléfonos o tabletas. Tampoco procesaremos pagos bancarios reales, publicaremos el sistema en Internet o pediremos autenticación obligatoria para consultar el catálogo.

3. Modelo general basado en dos procesos

Organizaremos el proyecto con los dos procesos de la tabla. De cada uno salen subprocesos concretos. Todos tienen una entrada, un trabajo definido, una persona responsable y una salida que podamos comprobar.

Código	Proceso principal	Propósito	Resultado
P1	Desarrollo iterativo	Entregar funciones útiles en incrementos cortos, probados y revisables por el cliente.	Incremento ejecutable, aceptado o devuelto con observaciones.
P2	Evolución y escalabilidad	Preparar cada versión para crecer sin perder estabilidad ni trazabilidad.	Versión optimizada, ampliable y lista para liberar.

Trabajaremos con P1 y P2 como dos procesos conectados. P1 dejará entregas parciales útiles desde las primeras iteraciones; P2 medirá cada versión y preparará su crecimiento. Al cerrar un incremento, el cliente revisará el resultado con el equipo. Lo que siga se decidirá con tres datos a la vista: valor de negocio, esfuerzo y riesgo.

4. Proceso P1 Desarrollo iterativo

En P1 cada incremento recorrerá análisis, diseño, código y prueba. El alcance será pequeño y quedará acordado antes de empezar. Al terminar, mostraremos una versión ejecutable al cliente, anotaremos sus observaciones y decidiremos si la entrega se acepta o vuelve a corrección.

5. Funcionamiento de las iteraciones de P1

1. Priorizamos: elegimos con el cliente el objetivo que aporta más valor, considerando el esfuerzo, el riesgo y las dependencias.

2. Analizamos: detallamos el alcance, los requisitos funcionales y no funcionales, y los criterios de aceptación del incremento.

3. Diseñamos y construimos: preparamos ventanas, datos e interfaces; después programamos e integramos la entrega.

4. Probamos y entregamos: verificamos funciones, datos, instalación y regresión antes de mostrar el incremento al cliente.

5. Escuchamos y ajustamos: registramos la retroalimentación, los defectos y las prioridades nuevas antes de comenzar el siguiente incremento.

La secuencia prevista será: incremento 1, catálogo y existencias; incremento 2, carrito y total; incremento 3, compra simulada y orden; incremento 4, instalación, respaldo y recuperación. Cada uno pasará por análisis, diseño, código y prueba, y terminará con una entrega revisable. El valor funcional crecerá con cada versión. Si un incremento no cumple su salida, volverá al subproceso responsable y no se convertirá en línea base.

6. Plan de incrementos y valor para el cliente

Incremento	Objetivo para el cliente	Recorrido interno	Entrega y revisión
1	Consultar catálogo, precio y existencia.	Análisis → Diseño → Código → Prueba	Catálogo funcional; el cliente revisa su utilidad y los datos mostrados.
2	Preparar la compra y conocer el total.	Análisis → Diseño → Código → Prueba	Carrito funcional; el cliente revisa la operación y el cálculo.
3	Confirmar la compra simulada y generar la orden.	Análisis → Diseño → Código → Prueba	Orden generada; el cliente revisa los datos y el flujo completo.
4	Instalar, respaldar y recuperar la información.	Análisis → Diseño → Código → Prueba	Versión instalable; cliente y equipo validan la operación y el cierre.

7. Proceso P2 Evolución y escalabilidad

P2 empieza cuando necesitamos saber si el programa seguirá funcionando bien al crecer. Revisamos el aumento de datos, funciones y equipos donde se instalará. Si hace falta mejorar algo, lo hacemos de forma controlada y cuidamos que las funciones ya aprobadas sigan comportándose igual.

8. Criterios técnicos de escalabilidad

- ✓ Separaremos la presentación, la lógica del negocio y el acceso a datos. Así podremos sustituir una parte sin reescribir las demás.
- ✓ Cada componente tendrá una responsabilidad clara y se comunicará mediante interfaces definidas.
- ✓ Usaremos consultas filtradas, índices y carga por bloques para no leer todo el catálogo o todas las órdenes de una sola vez.
- ✓ Dejaremos fuera del código las rutas, la conexión de datos, las copias de seguridad y los parámetros de instalación.
- ✓ Prepararemos migraciones controladas para pasar del almacenamiento local a una base compartida sin cambiar la lógica principal.
- ✓ Después de optimizar, refactorizar o migrar datos, volveremos a ejecutar las pruebas de regresión.

9. Correspondencia con las etapas oficiales

Aunque trabajaremos con P1 y P2, conservaremos los acrónimos oficiales porque indican en qué momento del ciclo estamos. Nos sirven como referencia. No los trataremos como ocho procesos separados.

10. Roles y responsabilidades

Para que el trabajo no quede sin dueño, asignamos cada decisión y cada entrega a un rol. Aun así, revisaremos los resultados en equipo cuando afecten a más de un área.

Acrónimo	Rol	Responsabilidades
AESA	Analista	Mantiene requisitos, casos de uso, alcance de iteración y trazabilidad.
DAOJ	Arquitecto	Define la modularidad, las interfaces y las decisiones de arquitectura y escalabilidad.
VMTS	Diseñador de BD y Desarrollador	Diseña los datos, construye componentes, mejora consultas y prepara migraciones.
KTCG	Calidad y Tester	Prepara y ejecuta pruebas, revisa salidas, atiende incidencias y autoriza líneas base.
Equipo de Desarrollo	Responsable colectivo	Integra, empaqueta e instala el programa. También mantiene el Manual Técnico.
Equipo del Proyecto	Responsable colectivo	Revisa los incrementos, prepara el Manual de Usuario y participa en el cierre.

11. Productos de trabajo

No todos los subprocesos terminan en código. También iremos dejando documentos, modelos y evidencias que expliquen qué hicimos y cómo lo comprobamos.

Artefacto	Contenido	Origen
EDOC	Estandarización de artefactos	P1.1
PDS	Esta guía de trabajo con P1 y P2	P1.1
SRS / CDU	Requisitos y casos de uso	P1.2
ARQ / DBD	Arquitectura modular y diseño de datos	P1.3 / P2.3

12. Herramientas y estándares

Usaremos herramientas conocidas por el equipo y guardaremos cada archivo en el formato que corresponde. La tabla aclara para qué ocuparemos cada una.

Herramienta	Uso autorizado
Microsoft Word	Documentos editables .docx y exportación oficial a PDF.
Microsoft Excel / TSPi	Estimaciones, tiempos, métricas y seguimiento.
StarUML / Draw.io	Diagramas UML, componentes y modelo entidad-relación.
Git / GitHub	Versiones del código, documentación y etiquetas de liberación.
IDE aprobado	Programación, depuración, construcción del ejecutable e instalador.

Seguiremos INV_EDOC_V1_1_190309_A y el Documento de Estandarización de Artefactos del Proyecto. IEEE 830 e ISO/IEC/IEEE 12207 quedarán como referencias técnicas. Si encontramos una diferencia entre los documentos sobre la tipografía, usaremos Times New Roman de 8 puntos en el Control de Versiones y en las tablas.

13. Criterios de entrada y salida de los subprocesos

Un subproceso empieza cuando su entrada está disponible. No lo daremos por terminado hasta contar con la salida indicada; si falta evidencia, el trabajo permanece abierto.

Subproceso	Entrada	Salida

Subproceso	Entrada	Salida
P1.1	Tenemos el objetivo y las prioridades.	Aprobamos alcance, responsables y tiempo.
P1.2	El alcance de la iteración está aprobado.	Los requisitos son claros, trazables y verificables.
P1.3	Los requisitos están aprobados.	El diseño modular es suficiente y ya fue revisado.
P1.4	Contamos con diseño y ambiente.	El ejecutable está integrado y versionado.
P1.5	La versión es estable y los CPR están listos.	Registramos resultados y cerramos los defectos críticos.

14. Requisitos y criterios de calidad de escritorio

Estos criterios nos dicen qué debe hacer el programa y qué nivel de calidad esperamos. Los comprobaremos con pruebas; una impresión general de que el sistema funciona no será suficiente.

Requisito	Criterio de aceptación
RF-01	Desde la ventana principal, el usuario puede consultar catálogo, precio y existencia sin iniciar sesión.
RF-02	Cuando agrega, modifica o elimina un producto, el total del carrito cambia de inmediato y conserva la precisión.
RF-03	Al confirmar, el programa valida los datos, registra la orden, vacía el carrito y muestra un número único.
RNF-01	Desde 1366 x 768, las ventanas, los textos, las tablas y los controles se ven completos. El programa funciona con teclado y mouse.
RNF-02	Si faltan datos o tienen un formato inválido, el programa detiene el registro, explica el problema y señala los campos que deben corregirse.
RNF-03	En el equipo de referencia, el catálogo y el carrito abren o se actualizan en 3 segundos como máximo, sin depender de Internet.
RNF-04	Podemos agregar un módulo mediante componentes e interfaces definidas sin modificar funciones que no están relacionadas.
RNF-05	Los datos permanecen después de cerrar el programa. El respaldo, la recuperación y las migraciones conservan su integridad.

15. Revisiones, cambios y configuración

Moveremos cada artefacto por tres estados: [BORRADOR], [EN REVISIÓN] y [APROBADO]. Antes de aprobarlo, KTCG comprobará el contenido, el formato, la trazabilidad y las salidas que le corresponden. La aprobación crea una línea base.

1. Registramos: anotamos el cambio, su motivo, la prioridad y los artefactos afectados.

2. Revisamos: medimos el impacto en P1, P2, los datos, la instalación, el tiempo y los riesgos.

3. **Decidimos:** aprobamos, rechazamos o dejamos el cambio para después, siempre con una razón registrada.
4. **Aplicamos:** actualizamos los artefactos, el código, las pruebas y la versión.
5. **Comprobamos:** ejecutamos la regresión y, cuando corresponde, fijamos una nueva línea base.

Controlaremos documentos, código, scripts de datos, recursos, configuración, ejecutables, instaladores y respaldos. Una línea base aprobada no se sobrescribe; cualquier cambio crea una versión nueva.

16. Trazabilidad e incidencias

Mantendremos esta cadena mínima: RF o RNF → CDU → componente o tabla → módulo o commit → CPR → resultado → incidencia. Así podremos seguir un requisito desde su origen hasta la prueba que lo comprueba. Cuando aparezca un problema, registraremos ID, fecha, versión, equipo, sistema operativo, módulo, requisito, pasos, resultado esperado y obtenido, severidad, responsable, estado, evidencia y cierre.

Severidad	Criterio	Tratamiento
Crítica	Impide instalar, iniciar o usar una función principal; compromete datos.	Corregir antes de continuar.
Alta	Afecta un requisito importante sin alternativa aceptable.	Corregir dentro de la iteración.
Media	Afectación parcial con alternativa temporal.	Planificar y verificar antes del cierre.
Baja	Detalle menor sin impacto funcional.	Priorizar según alcance y riesgo.

Usaremos estos estados: [ABIERTA], [ASIGNADA], [EN CORRECCIÓN], [LISTA PARA REPRUEBA], [CERRADA] y [RECHAZADA]. KTCG será quien cierre una incidencia, pero únicamente después de comprobar la corrección y ejecutar la regresión necesaria.

17. Métricas de iteración y escalabilidad

Mediremos lo necesario para decidir con datos, no por intuición. Además de tiempo y volumen, registraremos qué objetivo del cliente quedó aceptado en cada incremento. Las cifras guardarán liga con la versión y el equipo que hizo la prueba.

Métrica	Evidencia	Frecuencia	Responsable
Avance	Requisitos aceptados / requisitos de la iteración	Cada iteración	AESA

Métrica	Evidencia	Frecuencia	Responsable
Cobertura	Requisitos con CPR ejecutado / requisitos aplicables	P1.5 y P2.5	KTCG
Defectos	Abiertos, cerrados y reabiertos por severidad	Semanal	KTCG
Rendimiento	Tiempo de catálogo, carrito y consultas	P2.1 y P2.5	VMTS / KTCG
Volumen	Registros procesados sin degradación no aceptada	P2.5	VMTS / KTCG
Modularidad	Cambios realizados sin afectar módulos no relacionados	P2.3 a P2.5	DAOJ
Estimación	Minutos reales frente a minutos estimados	Por subproceso	Cada responsable
Valor entregado	Objetivos del cliente aceptados por incremento	Al cierre de cada incremento	AESA / KTCG

18. Riesgos y respuestas

Los riesgos de la tabla pueden aparecer en cualquier iteración. Si detectamos uno, aplicaremos la respuesta indicada y dejaremos evidencia de lo que se hizo.

ID	Riesgo	Respuesta	Responsable
R-01	Iteraciones demasiado grandes	Dividir por función y limitar trabajo en P1.1.	AESA / DAOJ
R-02	Dependencia entre módulos	Aplicar interfaces y revisión de arquitectura en P1.3 y P2.3.	DAOJ
R-03	Pérdida o corrupción de datos	Validar transacciones, respaldos y recuperación.	VMTS / KTCG
R-04	Rendimiento insuficiente	Medir, optimizar consultas y repetir P2.4-P2.5.	VMTS / KTCG
R-05	Mejoras que rompen funciones	Mantener pruebas base y ejecutar regresión completa.	KTCG

19. Entregables finales y cierre

- ✓ Entregaremos el programa de escritorio con catálogo, carrito, compra simulada y orden en funcionamiento.
- ✓ Incluiremos un ejecutable y un instalador probados en Windows 10 y Windows 11 de 64 bits.
- ✓ Dejaremos en el repositorio el código fuente, los scripts de datos, la configuración y la etiqueta de liberación.
- ✓ Cerraremos EDOC, PDS, SRS, CDU, ARQ, DBD, PPR, CPR, MTE, MUS e informe POS.
- ✓ Conservaremos la evidencia completa de P1 y P2, incluidas las métricas y las pruebas de regresión.

✓ Probaremos el respaldo y dejaremos documentados los pasos de recuperación y migración.

Podremos cerrar el proyecto cuando P1 y P2 hayan alcanzado sus salidas, la trazabilidad esté completa y no queden incidencias críticas o altas. KTCG deberá confirmar la línea base final.

20. Acrónimos y referencias

Acrónimo	Significado
STKD	Proyecto StockFlow de escritorio
P1	Proceso de Desarrollo Iterativo
P2	Proceso de Evolución y Escalabilidad
EDOC / PDS	Estandarización de artefactos / Proceso de desarrollo
SRS / CDU	Especificación de requisitos / Casos de uso
ARQ / DBD	Arquitectura / Diseño de base de datos
PPR / CPR	Plan de pruebas / Casos de prueba
MTE / MUS	Manual técnico / Manual de usuario
RF / RNF	Requerimiento funcional / no funcional

✓ INV_EDOC_V1_1_190309_A.pdf. Estándares para entrega de documentación.
✓ Documento de Estandarización de Artefactos del Proyecto de Empresa StockFlow.
✓ IEEE 830. Referencia para especificaciones de requisitos de software.
✓ ISO/IEC/IEEE 12207. Referencia para procesos del ciclo de vida del software.